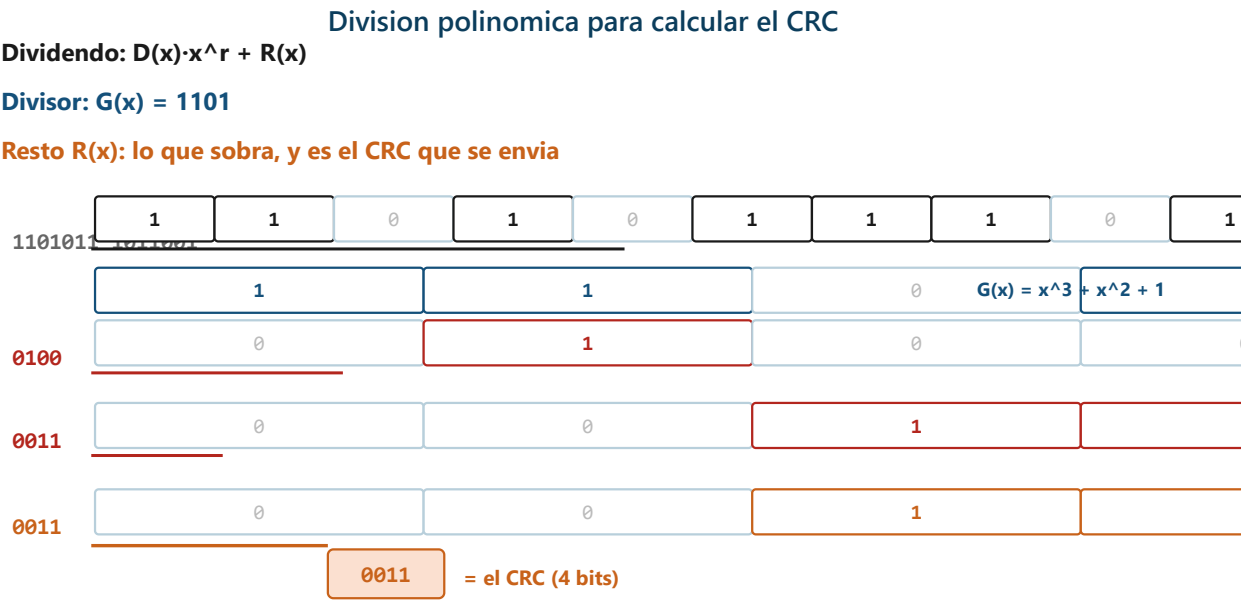
Clase 39 — Detección de errores: paridad, checksum y CRC

Bloque: E (Capa de Enlace) · **Semana:** 13 · **Duración estimada:** 120 min **Correlativa:** Clase 38 (servicios y entramado de la capa de enlace) · **Sigue con:** Clase 40 (corrección de errores y ARQ) **Material de referencia:** Tanenbaum cap. 4 · Kurose §5.5 · Práctico 5 de 2008 (ejercicios 1 a 4) · RFC 1071 · RFC 1624 · IEEE 802.3 FCS

🎯 Al terminar esta clase vas a poder

1. Explicar la diferencia entre **detectar** un error y **corregirlo**, y por qué un detector barato puede ser mejor que un corrector caro.
2. Calcular el **bit de paridad par** e **impar** de un mensaje, y **demostrar por qué la paridad detecta todos los errores de un número impar de bits y ninguno de un número par**.
3. Construir una **paridad en dos dimensiones** completa (matriz + paridades de fila + paridades de columna), **detectar y corregir un error de un bit**, y **demostrar por qué no puede corregir dos**.
4. Calcular el **Internet checksum** de 16 bits paso a paso, y explicar por qué **no detecta el intercambio de dos words**.
5. Calcular un **CRC** con **división polinómica** término a término en GF(2), y **verificarlo** con el método de **registro de desplazamiento con XOR** que usa el hardware.
6. Enunciar la **propiedad clave del CRC**: qué garantiza un generador de grado `r`, y por qué **el CRC detecta pero no corrige**.
7. Resolver completo el **Ejamen 2010-02 Pregunta 3** (8 puntos).



En GF(2) sumar es XOR y multiplicar es AND. Por eso la resta de la division es un XOR.

Se divide D(x)·x^r entre G(x) y se manda el resto pegado al mensaje original.

> **Prerequisito del examen:** `2010-02 P3` (8 pts — paridad en dos dimensiones: funcionamiento y cuántos bits corrige) · `2007-07 P2` (5 pts — nombre los mecanismos de detección y corrección de la capa de enlace y describa uno) · `2008-12 P4` (mismo tipo de comprobación, aplicado a POP3).
>
> ⚠️ Esta es **una clase de cálculo**. No hay nada que memorizar y sí mucho que ejecutar. Hacé los ejercicios con lápiz y papel antes de mirar la resolución.

PARTE 1 — TEORÍA

1.1 El modelo de error

El medio físico es **inexacto**: ruido, interferencia o crosstalk pueden hacer que un bit que salió como `1` llegue como `0`. Eso es un **error de bit** (*bit flip*).

```
Lo que se mandó:      1 0 1 1 0 0 1 0
Lo que llegó:        1 0 1 0 0 0 1 0
                          ↑ volteado
```

Sin protección, esa diferencia llega hasta la aplicación. **Detectar** errores es que el receptor **se dé cuenta** y descarte la trama. **Corregir** es que sepa **cuál era el bit original** y lo repare.

La diferencia es enorme en costo: un detector es barato (un registro, unos XOR y una división por un polinomio fijo); un corrector necesita **mucha** más redundancia, porque tiene que distinguir entre muchos patrones de error posibles.

La definición que ordena toda la tabla de §1.6:

> Un mecanismo detecta todos los errores de **hasta `d − 1` bits** y corrige todos los de **hasta `⌊(d − 1)/2⌋` bits**, donde `d` es la **distancia de Hamming mínima** del código: el mínimo número de bits en los que hay que cambiar cualquier par de mensajes válidos para que uno se convierta en el otro.

1.2 Chequeo de paridad de una dimensión (*parity bit*)

Un solo bit calculado sobre el mensaje, para que el número total de unos sea siempre el mismo.

```
Mensaje  D = 1 0 1 1 0 1 0        (7 bits, 4 unos)

Paridad PAR:   el total debe quedar PAR    → bit = 0   (4+0 = 4, par)
Paridad IMPAR: el total debe quedar IMPAR  → bit = 1   (4+1 = 5, impar)

Recepción: sumo los 8 bits (incluido el de paridad) y verifico que el total
           tenga la paridad esperada. Si no coincide → hubo error, descarto.
```

La limitación fundamental — y la demostración que hay que saber

> **La paridad detecta TODOS los errores de un número IMPAR de bits y NINGUNO de un número PAR de bits.**

Errores de número impar. Voltear `k` bits con `k` impar cambia el número de unos en `+k`, `+k−2`, …, `−k`: todos valores **impares**. La paridad del total se invierte, así que **no puede coincidir** con la guardada. Se detecta. ✓

Errores de número par. Voltear dos bits `i` y `j`:

```
los dos 1 → 0   :  los unos BAJAN en 2   → el total sigue siendo PAR
los dos 0 → 1   :  los unos SUBEN en 2   → el total sigue siendo PAR
uno a 1, otro a 0 : los unos NO cambian → el total no cambia
```

En los tres casos **la paridad del total no cambia**, así que el bit de paridad sigue cuadrando. El receptor **no se entera de nada**. ✗

```
Ejemplo:  D = 1 0 1 1 0 1 0 , paridad par = 0
Se voltean los bits 1 y 5 (ambos 1 → 0):
Se envió:  1 0 1 1 0 1 0 0        (6 unos = par ✓)
Se recibe: 0 0 1 1 0 1 0 0        (4 unos = par ✓)
→ NO DETECTADO. El receptor entrega un mensaje que nunca se mandó.
```

El costo. Un solo bit de paridad protege **una palabra** y agrega 1 bit: 12,5 % de overhead en un byte, 1 % en una trama de 100 bytes. Por eso **nadie usa un solo bit de paridad en un protocolo real**: el riesgo de que un error de dos bits pase inadvertido es demasiado alto. La solución es **múltiples bits de paridad**, y de ahí salen la paridad 2D y el código de Hamming (clase 40).

1.3 Paridad en dos dimensiones (*two-dimensional parity*)

La idea es repetir la paridad **en dos direcciones**.

```
Se ve el flujo como una MATRIZ:
 - cada palabra va en una FILA,
 - palabras consecutivas en filas consecutivas,
 - el bit j-ésimo de cada palabra forma la COLUMNA j-ésima.

Se agrega:  un bit de paridad al final de CADA FILA,
            un bit de paridad al final de CADA COLUMNA.
```

```
      c1 c2 c3 c4 | ... cn | Rp
┌──┬──────────────┬────────┬────┐
│f1│  n bits      │  ...   │    │  ← 1 bit de paridad por fila
├──┼──────────────┼────────┼────┤
│fm│  m × n       │        │    │
├──┼──────────────┼────────┼────┤
│Cp│  n bits      │        │    │  Cp = paridades de columna (última fila)
└──┴──────────────┴────────┴────┘

Campo de comprobación = (Rp(1)..Rp(m)) ‖ (Cp(1)..Cp(n))  =  m + n bits
```

Las dos propiedades que hacen que funcione:

1. **Detecta y corrige 1 error de bit.** Si falla la paridad de la fila `i` **y** la de la columna `j`, el error está **exactamente en `(i, j)`** — y se corrige, porque ya se sabe cuál es.
2. **Detecta los errores de 2 bits**, pero **no los puede corregir** (ver §2.2).

El detalle elegante de la esquina: no hace falta un bit de esquina, porque por construcción la paridad de la suma de las Rp y la de la suma de las Cp son **iguales** (las dos son la paridad del número total de unos

de la matriz). El campo mide `m + n` bits, no `m + n + 1`.

1.4 Internet checksum (suma de verificación de Internet)

Es el mecanismo de **IP, UDP y TCP**: un checksum de **16 bits** con **aritmética de complemento a uno** (*one's complement*) y acarreo circular.

Procedimiento, paso a paso (el que hay que saber de memoria):

```
1. PARTIR el mensaje en WORDS DE 16 BITS.
   Si el número de bytes es impar, se agrega un byte de relleno con 0.

2. SUMAR todas las words como enteros de 16 bits, en complemento a uno:
   suma normal, y cada vez que el resultado pasa de 65535 se le vuelve
   a sumar 1   →  "acarreo circular" o "end-around carry".

3. COMPLEMENTAR el resultado de 16 bits (invertir cada bit).

4. TRANSMITIR el mensaje seguido del checksum.

VERIFICACIÓN en el receptor: se suma todo (incluido el checksum) con la misma
aritmética. Si no hubo error, el resultado debe ser  0xFFFF.
```

Por qué detecta los errores — y el punto fino que suma puntos. El checksum es un **código lineal** sobre `GF(2)`. De ahí salen tres propiedades:

1. **Detecta el 100 % de los errores que están dentro de una sola word de 16 bits**, sea cual sea el patrón. Porque el valor de una word módulo $2^{16} - 1$ es **inyectivo**: dos words distintas nunca dan el mismo residuo. Si una word cambia, la suma cambia.
2. **Detecta todos los errores de un bit** (caso particular de lo anterior).
3. **NO detecta el intercambio de dos words de 16 bits**, porque **sumar es conmutativo**: $w_1 + w_2 = w_2 + w_1$. Si dos words se intercambian entre sí, la suma es **exactamente la misma**, el checksum es el mismo y el receptor acepta la trama.

Y el mismo argumento de la paridad, generalizado:

```
1 bit alterado, en una word   : la suma cambia ±2^k        (k=0..15) → ≠ 0 → DETECTADO
2 bits en la MISMA word       : cambia ±2^i ± 2^j (i≠j)               → ≠ 0 → DETECTADO
bit k en una word y bit k en otra, en sentidos opuestos:
                                +2^k y −2^k  → suma 0                 → NO DETECTADO
intercambio de 2 words        : la suma es idéntica                   → NO DETECTADO
```

Dónde vive esto. IP **no** tiene checksum (era demasiado caro recalcularlo en cada router); lo tienen **UDP y TCP**, porque corren en los extremos, donde unos pocos ciclos de CPU no importan y donde el datagrama puede haber pasado por veinte saltos. `RFC 1071` es la especificación del algoritmo y `RFC 1624` documenta por qué **cambiarlo rompe la compatibilidad** — un detalle que se pregunta.

1.5 CRC — Cyclic Redundancy Check

Esto es lo más importante de la clase. Todo lo anterior sirve para entender el problema; el CRC es la solución que se usa en todas partes.

1.5.1 La idea: polinomios y el campo GF(2)

La idea es representar el bloque de bits como un **polinomio con coeficientes binarios**. Si el bloque es `1 0 1 1 0`, el polinomio es:

```
1 0 1 1 0 → x⁴ + x² + x + x°
```

Porque el primer bit (el más a la izquierda) es el coeficiente de `x^(n-1)`.

Toda la aritmética se hace en el campo de Galois `GF(2)`, y `GF(2)` tiene **una sola diferencia con la aritmética normal**:

```
┌──────────────────────────────────────────────────┐
│ + es XOR (sin acarreo, 1+1 = 0)                  │
│ × es AND (0·x = 0, 1·x = x)                      │
└──────────────────────────────────────────────────┘

Ejemplos:
1 + 1 = 0
101 + 110 = 011 (XOR)
101 × 110 = 100 (AND)
```

Por qué polinomios y no "dividir el número entero": porque `GF(2)` no tiene división entera con resto. Los polinomios sobre `GF(2)` **sí** se pueden dividir, con resto, siempre. Esa es la razón matemática de que el CRC se defina como una división polinómica.

1.5.2 La división polinómica larga

El procedimiento completo, en palabras:

```
1. Sea G el polinomio generador, de grado r.
2. Sea D el bloque de datos de N bits. Se representa como polinomio.
3. Se agregan r CEROS a la derecha:  T = D · x^r
   (en binario: se desplazan D r posiciones hacia la izquierda).
4. Se divide T por G con "división larga", donde cada resta es un XOR.
5. El resto R (grado < r) es el CRC, y se transmite pegado a D.
```

El paso 4 en detalle — esto es lo que hay que escribir en el examen:

```
While el polinomio de trabajo T sea de grado >= grado de G:
    • Ubicar el término de MAYOR grado de T.
    • Poner G sobre él, desplazado para que coincidan las potencias mayores.
    • REEMPLAZAR esa porción de T por el XOR de ambas (o sea, "restar").
Cuando deg(T) < deg(G), lo que queda de T es el resto R.
```

Ejemplo completo, con `G = 1001 = x³+x+1` (grado 3) y `D = 10101010`. El desarrollo línea por línea, con los seis XOR, está en **§2.4**. Acá va la estructura, que es lo que hay que entender:

```
D · x³ = 10101010 000 = 10101010000        (grado 10)

x¹⁰ → ⊕ G en x¹⁰  →  00111010000
x⁸  → ⊕ G en x⁸   →  00011110000
x⁷  → ⊕ G en x⁷   →  00001100000
x⁶  → ⊕ G en x⁶   →  00000101000
x⁵  → ⊕ G en x⁵   →  00000000110
x³  → ⊕ G en x³   →  00000000101   ← grado 2 < 3, FIN

R = 101 = 5 = 0x5
```

Comprobación (multiplicación de vuelta en `GF(2)`):

```
T ⊕ R = 10101010101 ⊕ 00000000101 = 10101010000 = D · x³  ✓
Y 10101010101 = 1001 × 10111101   →  divisible, resto 0  ✓
```

Lo que se transmite es `D || R`:

```
10101010 101
└──D────┘ └R┘   8 + 3 = 11 bits para 8 de información
overhead = 3 bits = el grado de G
```

1.5.3 El cálculo con XOR: cómo lo hace realmente el hardware

Nadie implementa una división polinómica con restas. El hardware real usa un **registro de desplazamiento de `r` bits** y una sola operación de XOR:

```
R = 0 (r bits)
for cada bit b del mensaje D, del más significativo al menos:
fb = (bit más significativo de R) XOR b
R = (R << 1), quedándose con r bits
si fb = 1: R = R XOR (G módulo 2^r)
Al final, R es el CRC.
```

Por qué es equivalente a la división polinómica: cada iteración hace exactamente **una** de las "restas" (XOR) de la división larga, y lo hace en el momento en que el bit que sale por la izquierda del registro corresponde al término mayor. Al final, en el registro queda lo mismo que en el polinomio de trabajo: el resto. La ventaja es que **es un circuito de `r` flip-flops, `r` puertas XOR y un cable de realimentación** — o sea, unos pocos nanoseconds por byte.

La tabla completa del ejemplo anterior (`G = 1001`, `r = 3`, `D = 1 0 1 0 1 0 1 0`) está en **§2.5** y da `R = 101`, el mismo resultado que la división polinómica. Eso es exactamente lo que hay que demostrar cuando alguien dice "el XOR de hardware es equivalente a la división", y podés decirlo en el examen poniendo los dos resultados de lado.

1.5.4 Los polinomios generadores estándar, y por qué hay tantos

Nombre	Polinomio	Dónde se usa
CRC-32 (IEEE 802.3)	0x04C11DB7 grado 32	Ethernet (IEEE 802.3), ZIP, PNG, gzip, HTTP
CRC-32C (Castagnoli)	0x1EDC6F41 grado 32	SCTP, iSCSI, ext4, Ceph
CRC-16/CCITT	0x1021 = $x^{16}+x^{12}+x^5+1$	HDLC (FCS), X.25, MPEG-2, Bluetooth
CRC-8	0x07 = x^8+x^2+x+1	ATM (celdas de 53 bytes)

En el hardware se usan las versiones "reflejadas" (*reflected*): el registro desplaza a la derecha en vez de a la izquierda, y los polinomios aparecen invertidos byte a byte. Por eso el CRC-32 se escribe `0xEDB88320` en el software y `0x04C11DB7` en el estándar, y el CRC-32C aparece como `0x82F63B78`.

¿Por qué hacen falta polinomios distintos? Por cuatro razones, todas de optimización:

1. **Detectar todos los errores de un bit** ⇒ el término independiente tiene que ser **1** (el polinomio no puede ser divisible por `x`).
2. **Detectar ráfagas cortas** ⇒ el polinomio debe tener **muchos términos no nulos** y ser irreducible o primitivo, para que la división produzca muchos restos distintos.
3. **Detectar el error extremo a extremo además del de línea**: en una llamada telefónica, dos FCS de línea iguales pueden **compensarse** y anular un error; un polinomio distinto por capa lo evita.
4. **Ser barato en hardware** ⇒ se elige el grado que da el overhead justo (32 bits = 4 bytes, comparable de un vistazo).

Lo que se escribe en el examen es la idea: *el mismo problema, optimizado para contextos distintos, tiene soluciones distintas; por eso hay varios polinomios estandarizados y no uno solo.*

1.5.5 La propiedad clave del CRC — y el punto conceptual que se pregunta

Acá está todo. Sea `G` un polinomio de grado `r`.

> **Un patrón de error `E` NO es detectado si y sólo si `G` divide a `E`.** En otras palabras: los patrones no detectados son exactamente **los múltiplos de `G`**.

De ahí salen las tres garantías:

(a) Se detecta el 100 % de los errores de UN bit.

```
Un error de un solo bit en la posición k es el polinomio E = x^k.
Para que fuera undetected, haría falta G | x^k.
Pero x^k tiene grado k, y el coeficiente de x^k en G... un polinomio de
grado r ≥ 1 no divide a un monomio, porque x^k tiene un solo término.
∴ E ≠ 0 nunca es undetected.
```

(b) Se detectan TODOS los errores por ráfaga (*burst error*) de longitud ≤ r.

```
Una ráfaga de longitud L es un error cuyos bits están todos en una ventana
de L posiciones consecutivas: E = x^s · (polinomio de grado ≤ L−1)
Si L ≤ r, entonces grado(E) ≤ s + r − 1 ... y el factor de la derecha tiene
grado < r = grado(G), así que NO puede ser múltiplo de G.
∴ undetected imposible. ✓
```

Ésta es la razón por la que el CRC anduvo en enlaces de datos reales durante décadas: los errores de línea **no son aleatorios**, son **ráfagas** (un transitorio electromagnético voltea un grupo de bits juntos), y el CRC los detecta todos mientras sean cortos.

(c) Para un error de `b` bits, la fórmula y su lectura correcta.

```
Los undetected son los múltiplos de G: E = G · Q.
En una ventana de b posiciones, los múltiplos de G son los G·Q con
deg(Q) ≤ b−1−r, o sea 2^(b−r) cocientes distintos.

De ahí sale la expresión que circula en los apuntes:

P(detectar) = 1 − 2^(−(n − b))

donde n = grado del polinomio generador y b = número de bits del error.

⚠️ CÓMO HAY QUE USARLA: la expresión da sentido numérico sólo mientras
b sea mucho menor que n. Interpretada así:

b = 1 → P ≈ 1 − 2^(−n) ≈ 1 (seguro)
b = n/2 → P ≈ 1 − 2^(−n/2) ≈ 1 (muy seguro)
b ≈ n → la protección se agota

⚠️ La versión RIGUROSA — y la que hay que saber argumentar — son (a), (b) y
la distancia mínima:

d = r + 1 (para los generadores bien elegidos, sin múltiplos
de peso bajo)

∴ se detectan TODOS los errores de peso ≤ r = d − 1.

Los errores de peso mayor se detectan con probabilidad muy alta, pero no
con certeza: existen C(n, b) patrones posibles y sólo 2^(b−r) de ellos son
múltiplos de G.
```

Consecuencia práctica, que es lo que se lleva uno al examen:

```
CRC-32, r = 32:
error de 1 bit → detectado SIEMPRE
ráfaga de ≤ 32 bits → detectada SIEMPRE
error de 32 bits → probabilidad de detección ≈ 1 − 2^(−32)
error de 64 bits → todavía altísima
(y en la práctica, más de 10^11 bits transferidos con undetected = 0)
```

1.5.6 ⚠️ El CRC detecta, NO corrige

Esto se pregunta, y la respuesta tiene dos partes:

1. Teóricamente, un CRC con `d = r + 1` podría corregir `⌊r/2⌋` errores. Con `d = 33`, serían 16. ¿Por qué no se hace?

- El **síndrome** (el resto de la división) tiene sólo `r = 32` bits. El número de patrones de error de **un** bit es `N`, mucho mayor que `2^32`. Con una tabla de log se puede localizar un error de un solo bit (`G·x^i` módulo `x^r` da la tabla), pero para 16 errores hay `binom(N,16)` patrones y **no hay forma práctica de distinguirlos** con 32 bits de información.
- El hardware real de CRC hace una sola cosa: comparar el resto con cero.

2. Para corregir de verdad se usan otros códigos: Hamming y SECDED (clase 40, para 1 y 2 bits) y los **códigos correctores de Reed–Solomon** o **LDPC**, que agregan mucha más redundancia y son los que se usan en discos, DVDs, satelite, DVB-T y 5G.

> **La frase para el examen:** *el CRC es un mecanismo de detección de alta probabilidad con overhead mínimo. La corrección exige redundancia considerable y algoritmos de decodificación distintos; por eso nunca se usa el CRC como corrector.*

1.6 La tabla que resume todo

Mecanismo	Redundancia	Detecta	Corrige	Dónde
Paridad 1D	1 bit	**errores de peso impar**; falla con peso par	nada	xo, casi nunca se usa solo
Paridad 2D	`m + n` bits	**todos** los de 1 y 2 bits; **no** todos los de 4	**1 bit**	práctica, teaching
Internet checksum	16 bits	todos los de 1 bit; **casi todos** los múltiples; **NO** word swaps	nada	IP(no), **UDP, TCP**
Hamming SEC	`r` bits, `2^r ≥ m+1`	hasta `d−1`	**1 bit**	clase 40
Hamming SECDED	`r + 1`	2 bits	**2 bits**	clase 40
CRC-r	`r` bits	**1 bit siempre**, ráfagas ≤ `r` siempre, más	**nada**	**Ethernet, ZIP, PNG, iSCSI, SCTP**
Reed–Solomon	alto	ráfagas largas	**varios bits**	DVD, DVB-T, 5G, RAID 6

1.7 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Paridad**: par e impar, y la propiedad de detectar todo error de **número impar** de bits y ninguno de **número par**. - **Internet checksum**: 16 bits, complemento a uno, con acarreo circular. Vive en **UDP y TCP**. Lo que **NO** detecta: el **intercambio de dos words**. - **CRC**: división polinómica en `GF(2)` con `+` = XOR, `r` ceros agregados, y la propiedad "detecta siempre un error de 1 bit y toda ráfaga de hasta `r` bits". - **CRC-32 =** `0x04C11DB7` para Ethernet, ZIP y PNG. **CRC detecta, no corrige.**

Cambió o se afinó: - **El campo de aplicación se movió, no el mecanismo.** `CRC-32C` (Castagnoli) es el de **SCTP, iSCSI, ext4 y Ceph**; `CRC-32` es el de **Ethernet, ZIP y PNG**. Conviven por las cuatro razones de §1.5.4. - **El CRC dejó de ser un cálculo**: en 1 a 400 Gbit/s se implementa como **circuito combinacional** (tabla de 256 entradas de 32 bits, o una matriz lineal de XOR y desplazamientos). El resultado es el mismo. - **El checksum de IPv4 no existe y no volvió**: los routers lo descartan para ganar velocidad. `RFC 1624` documenta que **no se puede cambiar** el algoritmo del checksum de TCP/UDP sin romper compatibilidad. - ⚠️ **Tema en desuso:** la **paridad 2D como mecanismo de producción**. Se enseña y se pregunta, pero no aparece en ningún protocolo comercial: todo lo real usa CRC o **Reed–Solomon**. - Lo que **sí** se usa para **corregir** de verdad: **Hamming** y **SECDED** (clase 40), y **Reed–Solomon** o **LDPC** en discos, DVB-T, 5G y RAID 6.

Consecuencia para el examen: la pregunta `2010-02 P3` **se sigue preguntando tal cual**. Lo que suma es el criterio de **distancia mínima**, que es lo que distingue a quien entendió de quien memorizó.

PARTE 2 — PRÁCTICA RESUELTA

> Los **ejercicios 1 a 4** son del **Práctico 5 de 2008** (`material-oficial/practicos/png/prac05_2008-p01.png`), extraídos del libro de Kurose & Ross. Los **ejercicios 5 a 11** son *inventados y resueltos*.

2.1 Ejercicio 1 (Práctico 5) — Paridad en dos dimensiones con paridad par

Enunciado. Supongamos que el contenido de un paquete es el patrón de bits `1010101010101011`, y que se utiliza un esquema de **paridad par**. ¿Cuál sería el valor del campo de comprobación para el caso de un esquema de paridad de **dos dimensiones**? La respuesta debe usar un campo de comprobación de **longitud mínima**.

Paso 1 — Dimensionar la matriz

Son 16 bits de datos. Con una matriz de `m` filas de `n` columnas hay `m · n` posiciones de datos, y el campo de comprobación mide `m + n` bits (un bit de paridad por fila + uno por columna; la esquina no aporta nada, §1.3).

Buscamos el mínimo de `m + n` con `m · n ≥ 16`:

```
m n m·n m+n
─── ─── ──── ────
4 4 16 8 ← mínimo
5 4 20 9
3 6 18 9
2 8 16 10
1 16 16 17
8 2 16 10
```

Matriz de 4 × 4. la convención de la cátedra ("el bit j-ésimo de cada palabra forma la columna j-ésima"), el patrón se parte en cuatro palabras de cuatro bits:

```
f1 = 1010
f2 = 1010
f3 = 1010
f4 = 1011
```

Paso 2 — Paridad de cada FILA

Paridad par ⇒ el bit de paridad vale **(número de unos) mod 2**.

```
f1 = 1 0 1 0 → 2 unos → 2 mod 2 = 0
f2 = 1 0 1 0 → 2 unos → 0
f3 = 1 0 1 0 → 2 unos → 0
f4 = 1 0 1 1 → 3 unos → 3 mod 2 = 1
Rp = 0 0 0 1
```

Paso 3 — Paridad de cada COLUMNA

```
c1 (bit 1 de cada palabra): 1,1,1,1 → 4 unos → 0
c2 (bit 2): 0,0,0,0 → 0 unos → 0
c3 (bit 3): 1,1,1,1 → 4 unos → 0
c4 (bit 4): 0,0,0,1 → 1 uno → 1
Cp = 0 0 0 1
```

Respuesta

```
c1 c2 c3 c4 | Rp
┌──┬─────────┬──┐
│f1│ 1 0 1 0 │ 0 │
│f2│ 1 0 1 0 │ 0 │
│f3│ 1 0 1 0 │ 0 │
│f4│ 1 0 1 1 │ 1 │
├──┼─────────┼──┤
│Cp│ 0 0 0 1 │  │
└──┴─────────┴──┘

CAMPO DE COMPROBACIÓN = Rp || Cp = 0 0 0 1 0 0 0 1 = 8 bits

Frame total = 16 bits de datos + 8 de comprobación = 24 bits.
```

Verificación de la esquina (la propiedad de §1.3): la paridad de la suma de las paridades de fila es `(0+0+0+1) mod 2 = 1`; la de la suma de las de columna es `(0+0+0+1) mod 2 = 1`. **Coinciden**, como debe ser.

2.2 Ejercicio 2 (Práctico 5) — 2D paridad: corrige 1 bit, detecta 2

Enunciado. Mostrar, con un ejemplo **distinto al del libro**, que la paridad de dos dimensiones puede **corregir y detectar un error en un bit único**. Mostrar, con otro ejemplo, que un error en **dos** bits puede ser **detectado pero no corregido**.

(a) Corrección de un error de un bit

Datos (paridad par), matriz de 4 × 4:

```
      c1 c2 c3 c4 | Rp
 f1   1  0  1  1 |  1     ← 3 unos
 f2   0  1  1  0 |  0     ← 2 unos
 f3   1  1  0  0 |  0     ← 2 unos
 f4   0  0  1  0 |  1     ← 1 uno
 ─────────────────┼──
 Cp   0  0  1  1 |         ← c1:2→0  c2:2→0  c3:3→1  c4:1→1
```

Se voltea el bit de la fila 3, columna 3 (de `0` a `1`). El receptor recalcula:

```
FILAS
 f1 = 1 0 1 1 → 3 unos → Rp debe ser 1. Recibido 1. ✓
 f2 = 0 1 1 0 → 2 unos → Rp debe ser 0. Recibido 0. ✓
 f3 = 1 1 1 0 → 3 unos → Rp debe ser 1. Recibido 0. ✗  ← FALLA
 f4 = 0 0 1 0 → 1 uno  → Rp debe ser 1. Recibido 1. ✓

COLUMNAS
 c1 = 1,0,1,0 → 2 unos → Cp debe ser 0. Recibido 0. ✓
 c2 = 0,1,1,0 → 2 unos → Cp debe ser 0. Recibido 0. ✓
 c3 = 1,1,1,1 → 4 unos → Cp debe ser 0. Recibido 1. ✗  ← FALLA
 c4 = 1,0,0,0 → 1 uno  → Cp debe ser 1. Recibido 1. ✓
```

```
Fila que falla = 3 ; Columna que falla = 3
∴ el bit equivocado es el de la intersección (3,3)
∴ CORRIGE: se lo invierte y el mensaje queda intacto.  ✓
```

Interpretación. Ese es el corazón de la técnica: la fila te dice **en qué franja** y la columna te dice **en qué posición dentro de la franja**. Con esos dos datos la posición del error queda **completamente determinada**: no hay ambigüedad posible.

(b) Dos errores: detecta, pero no corrige

Con la misma matriz, se voltean los bits **(1,1)** y **(2,2)**:

```
FILAS
 f1 = 0 0 1 1 → 2 unos → Rp debe ser 0. Recibido 1. ✗  ← FALLA
 f2 = 0 0 1 0 → 1 uno  → Rp debe ser 1. Recibido 0. ✗  ← FALLA
 f3 = 1 1 0 0 → 2 unos → 0. Recibido 0. ✓
 f4 = 0 0 1 0 → 1 uno  → 1. Recibido 1. ✓

COLUMNAS
 c1 = 0,0,1,0 → 1 uno  → Cp debe ser 1. Recibido 0. ✗  ← FALLA
 c2 = 0,0,1,0 → 1 uno  → Cp debe ser 1. Recibido 0. ✗  ← FALLA
 c3 = 1,1,0,1 → 3 unos → 1. Recibido 1. ✓
 c4 = 1,0,0,0 → 1 uno  → 1. Recibido 1. ✓
```

```
Filas que fallan    : 1 y 2
Columnas que fallan: 1 y 2

Posiciones candidatas = las 4 intersecciones:
      (1,1)  (1,2)
      (2,1)  (2,2)
y SOLO dos están mal.  ¿Cuáles?  ¡INDETERMINADO!

∴ DETECTA (hay incoherencia) pero NO PUEDE CORREGIR.
```

Por qué no puede: la información no alcanza. El receptor tiene cuatro observaciones ("fallan estas dos filas y estas dos columnas") y hay **cuatro** combinaciones distintas de dos bits que las producen exactamente igual: `(1,1)+(2,2)`, `(1,1)+(2,1)`, `(1,2)+(2,1)` y `(1,2)+(2,2)`. **Sin redundancia extra no hay forma de desempatar.**

(c) Y el caso peor: un error de CUATRO bits que **no se detecta**

Volteamos un **rectángulo 2 × 2**: (1,1), (1,2), (2,1) y (2,2).

```
FILAS
 f1 = 0 1 1 1 → 3 unos → Rp = 1. Recibido 1. ✓   (original: 1 0 1 1 = 3 unos)
 f2 = 1 0 1 0 → 2 unos → Rp = 0. Recibido 0. ✓   (original: 0 1 1 0 = 2 unos)
 f3 = 1 1 0 0 → 2 unos → 0. Recibido 0. ✓   (sin cambios)
 f4 = 0 0 1 0 → 1 uno  → 1. Recibido 1. ✓   (sin cambios)

COLUMNAS
 c1 = 0,1,1,0 → 2 unos → 0. Recibido 0. ✓   (original: 1,0,1,0 = 2 unos)
 c2 = 1,0,1,0 → 2 unos → 0. Recibido 0. ✓   (original: 0,1,1,0 = 2 unos)
 c3 = 1,1,0,1 → 3 unos → 1. Recibido 1. ✓   (sin cambios)
 c4 = 1,0,0,0 → 1 uno  → 1. Recibido 1. ✓   (sin cambios)
```

Resultado: CERO verificaciones fallan. El receptor acepta una trama con **cuatro** bits equivocados y no se entera de nada. Ése es el tope de la paridad 2D, y por eso no se usa en producción.

La razón, en una frase: cada fila y cada columna afectada recibe **dos** cambios de signo opuesto, que se **cancelan** en el conteo de unos. Exactamente el mismo argumento de la paridad 1D, pero aplicado a las dos dimensiones a la vez.

⚠️ Errores típicos

- **Calcular la paridad de columna leyendo las filas.** La columna `j` es el bit `j` de **cada** fila, no el último bit de cada fila.
- **Olvidar que la paridad es la del número de unos**, no "la del último bit".
- **Cerrar en 8 bits el campo de comprobación cuando `m·n > 16`.** Si la matriz no es cuadrada, el campo mide `m + n`, y hay que elegir la que minimiza.
- **Confundir "detectar 2 errores" con "corregir 2 errores".** Con dos filas y dos columnas fallando hay cuatro candidatos y sólo dos están mal.

2.3 Ejercicio 3 (Práctico 5) — Internet checksum de los bytes 0 a 9

Enunciado. Supóngase que la porción de información de un paquete contiene diez bytes, que suponen la representación binaria sin signo de ocho bits de los enteros del 0 al 9. Calcular la suma de comprobación (*checksum*) Internet para estos datos.

Paso 1 — Los diez bytes en binario y en hexadecimal.

```
decimal binario hex
_______ _________ ____
0 00000000 0x00
1 00000001 0x01
2 00000010 0x02
3 00000011 0x03
4 00000100 0x04
5 00000101 0x05
6 00000110 0x06
7 00000111 0x07
8 00001000 0x08
9 00001001 0x09
```

Paso 2 — Emparejarlos en words de 16 bits (un byte par, un byte impar).

```
byte: 0 1 | 2 3 | 4 5 | 6 7 | 8 9
↓ ↓ | ↓ ↓ | ↓ ↓ | ↓ ↓ | ↓ ↓
0x0001 | 0x0203 | 0x0405 | 0x0607 | 0x0809

w1 = 0000 0000 0000 0001 = 0x0001 = 1
w2 = 0000 0010 0000 0011 = 0x0203 = 515
w3 = 0000 0100 0000 0101 = 0x0405 = 1029
w4 = 0000 0110 0000 0111 = 0x0607 = 1543
w5 = 0000 1000 0000 1001 = 0x0809 = 2057
```

Paso 3 — Sumar. Ojo: **10 bytes son 5 words exactas, así que no hace falta byte de relleno** (el relleno con 0 sólo hace falta cuando el número de bytes es impar).

```
S = 1 + 515 + 1029 + 1543 + 2057

1 + 515 = 516
516 + 1029 = 1545
1545 + 1543 = 3088
3088 + 2057 = 5145

5145 < 65535 → NO hay acarreo que devolver. S = 5145
```

Paso 4 — Complementar los 16 bits.

```
5145 en binario: 0001 0100 0001 1001
| | | |
Su complemento: 1110 1011 1110 0110
= 0xEBE6

Decimales: 65535 − 5145 = 60390 = 0xEBE6
```

Respuesta

```
Internet checksum = 0xEBE6 = 60390 (16 bits)

Trama completa: 00 01 02 03 04 05 06 07 08 09 | EB E6
```

Interpretación. Con **diez bytes** el checksum es de **2 bytes**. La proporción de protección es del 20 %: muchísima redundancia para tan pocos datos, que es exactamente lo que pasa con los mensajes cortos. Los checksums tienen sentido en los encabezados de **TCP/IP** (decenas de bytes), no en un payload chico.

⚠️ Errores típicos

- **Sumar byte a byte y no word a word.** Hay que agrupar de a 16 bits, y el **primer byte va en el bit más alto** de la word.
- **Olvidar el acarreo circular.** Acá no hubo, pero en el ejercicio 2.8 sí.
- **Complementar dos veces** (una al sumar, otra al final).

2.4 Ejercicio 4 (Práctico 5) — CRC con `G = 1001` y `D = 10101010`

Enunciado. Considérese el generador de cuatro bits `G`, mostrado en la Figura 5.8 del libro, y supóngase que `D` tiene el valor `10101010`. ¿Cuál es el valor de `R`?

```
G = 1001 = x³ + x + 1 grado r = 3
D = 10101010 = x⁷+x⁵+x³+x¹ grado 7
```

El desarrollo completo de la división polinómica está hecho en §1.5.2, con los seis pasos y todos los XOR. Se reproduce acá en forma de tabla, que es como hay que presentarlo:

```
T = D·x³ = 10101010 000 = 10101010000 (11 bits)

iter término mayor XOR con G alineado T después
──── ──────────── ──────────────────────── ──────────
1 x¹⁰ 10010000000 (x¹⁰+x⁸+x⁷) 00111010000
2 x⁸ 00001001000 (x⁸+x⁶+x⁵) 00011110000
3 x⁷ 00000010010 (x⁷+x⁵+x⁴) 00001100000
4 x⁶ 00000001001 (x⁶+x⁴+x³) 00000101000
5 x⁵ 00000000110 (x⁵+x³+x²) 00000000110
6 x³ 00000000010 (x³+x+1) 00000000101
──────────
grado 2 < 3 → R = 101
```

El paso a paso, línea por línea (esto es lo que se escribe en el examen):

```
Paso 1 1 0 1 0 1 0 1 0 0 0 0
⊕ 1 0 0 1 0 0 0 0 0 0 0
─────────────────────
0 0 1 1 1 0 1 0 0 0 0

Paso 2 0 0 1 1 1 0 1 0 0 0 0
⊕ 0 0 0 1 0 0 1 0 0 0 0
─────────────────────
0 0 0 1 1 1 1 0 0 0 0

Paso 3 0 0 0 1 1 1 1 0 0 0 0
⊕ 0 0 0 1 0 0 0 1 0 0 0
─────────────────────
0 0 0 0 1 1 0 0 0 0 0

Paso 4 0 0 0 0 1 1 0 0 0 0 0
⊕ 0 0 0 0 1 0 0 1 0 0 0
─────────────────────
0 0 0 0 0 1 0 1 0 0 0

Paso 5 0 0 0 0 0 1 0 1 0 0 0
⊕ 0 0 0 0 0 1 0 0 1 0 0
─────────────────────
0 0 0 0 0 0 0 1 1 0 0

Paso 6 0 0 0 0 0 0 0 1 1 0 0
⊕ 0 0 0 0 0 0 0 1 0 0 1
─────────────────────
0 0 0 0 0 0 0 0 1 0 1
R = 1 0 1
```

Respuesta

```
R = 101 (3 bits = el grado de G)
Trama que va por el cable: 10101010 101
└─ D ──┘ └R┘
Receiver: 10101010101 ÷ 1001 = 10111101, resto 0 ✓
```

> **Nota sobre la numeración del Práctico 5.** El PDF oficial (`prac05_2008-p01.png`) numera este ejercicio como el **número 4**, y al CRC como el **4** y el checksum como el **3**. Algunas reediciones lo numeran como "Ejercicio 2". Acá se usa la numeración del PDF de FING.

2.5 Verificación del Ejercicio 4 con el método de XOR

Mismo dato, mismo generador, método de hardware. La tabla completa está en §1.5.3 y da `R = 101`. Se reproduce para que se vea el paralelismo:

```
G = 1001, r = 3, G mod 2³ = 001
D = 1 0 1 0 1 0 1 0

paso b R antes MSB⊕b R<<1 ¿XOR 001? R final
──── ─── ──────── ────── ─────── ────────── ────────
1 1 000 1 000 sí 001
2 0 001 0 010 no 010
3 1 010 1 100 sí 101
4 0 101 1 010 sí 011
5 1 011 1 110 sí 111
6 0 111 1 110 sí 111
7 1 111 0 110 no 110
8 0 110 1 100 sí 101

R = 101 ✓ IGUAL
```

Interpretación. Los dos métodos dan `101`. Eso es lo que significa "son equivalentes": **el registro de desplazamiento con XOR hace exactamente la misma división polinómica**, pero con `r` flip-flops en vez de una tabla.

2.6 Ejercicio 5 (inventado) — Paridad par e impar sobre el mismo mensaje

Datos. `D = 11010011` (8 bits). Número de unos = **5**.

```
Paridad PAR:   5 es impar → el bit debe hacer el total PAR    → bit = 1
               (5 + 1 = 6, par ✓)
Paridad IMPAR: 5 es impar → el total debe quedar IMPAR        → bit = 0
               (5 + 0 = 5, impar ✓)
```

Comprobación con los dos errores de un bit posibles:

```
Se envía: 1 1 0 1 0 0 1 1 | 1     (paridad par;  6 unos = par)
Error en bit 3 (0→1):  ahora 7 unos → IMPAR
                      el receptor con paridad par espera PAR → NO CUADRA → DETECTADO ✓
Error en bit 5 (0→1):  idéntico → DETECTADO ✓

Se envía: 1 1 0 1 0 0 1 1 | 0     (paridad impar; 5 unos = impar)
Error en bit 3:         → 6 unos → PAR; la paridad impar espera IMPAR → DETECTADO ✓
```

Y el caso que NO detecta:

```
Se envía: 1 1 0 1 0 0 1 1 | 1
Se voltean los bits 3 y 5 (los dos 0 → 1): quedan 8 unos = PAR
→ el receptor ve 8 unos, que es PAR, y espera PAR → NO DETECTADO ✗
```

2.7 Ejercicio 6 (inventado) — Paridad 2D: cuatro escenarios con la misma matriz

Se usa la matriz de §2.2(a), con el bit (3,3) correcto:

```
      c1 c2 c3 c4 | Rp
 f1   1  0  1  1 |  1
 f2   0  1  1  0 |  0
 f3   1  1  0  0 |  0
 f4   0  0  1  0 |  1
 Cp   0  0  1  1 |
```

Escenario A — un error en (3,3).

```
Fallan f3 (3 unos, esperaba 2) y c3 (4 unos, esperaba 3)
→ intersección (3,3) → CORRIGE. ✓
```

Escenario B — dos errores en (1,1) y (4,4).

```
f1 = 0 0 1 1 → 2 unos → Rp 0, hay 1  ✗ FALLA
f4 = 0 0 1 1 → 2 unos → Rp 0, hay 1  ✗ FALLA
c1 = 0,0,1,0 → 2 unos → Cp 0, hay 0  ✓
c4 = 1,1,0,1 → 3 unos → Cp 1, hay 1  ✓
→ 2 filas y 0 columnas. Detecta, no corrige. ✗
```

Escenario C — dos errores en (1,1) y (2,1), en la misma columna.

```
f1 = 0 0 1 1 → 2 unos → Rp 0, hay 1  ✗ FALLA
f2 = 1 1 1 0 → 3 unos → Rp 1, hay 0  ✗ FALLA
c1 = 0,1,1,0 → 2 unos → Cp 0, hay 0  ✓   (¡la columna NO falla: los dos
                                           errores se compensan!)
→ 2 filas y 0 columnas. Detecta, no corrige. ✗
```

Escenario D — dos errores en (1,1) y (2,2).

```
Fallan f1, f2, c1, c2  (dos filas, dos columnas)
→ Detecta, no corrige.  (Es el caso de §2.2(b).)
```

Conclusión del ejercicio:

Filas que fallan	Columnas	Qué puede hacer
1	1	CORRIGE
2	2	detecta, no corrige
2	0	detecta, no corrige
0	0	¡nada! (4 errores)

2.8 Ejercicio 7 (inventado) — Internet checksum con acarreo circular

Datos. Cuatro words de 16 bits: `0x1234 = 4660`, `0x5678 = 22136`, `0x9ABC = 39612`, `0xDEF0 = 57072`.

Paso 1 — sumar y arrastrar el acarreo.

```
    4660
  + 22136   →  26796   (< 65536, sin acarreo)
  + 39612   →  66408   (≥ 65536 → hay acarreo)

    66408  −  65536  =    872
    872    +      1  =    873      ← acarreo devuelto al final

  + 57072   →  57945   (< 65536, sin acarreo)

S = 57945
```

Paso 2 — complementar.

```
~57945 = 65535 − 57945 = 7590
7590 = 1×4096 + 13×256 + 10×16 + 6 = 4096 + 3328 + 160 + 6  → 0x1DA6
```

Paso 3 — verificar.

```
57945 + 7590 = 65535 = 0xFFFF  ✓   (todo unos: checksum correcto)
```

Respuesta: Internet checksum = `0x1DA6` = 7590.

El caso que NO detecta, demostrado con estos números:

```
Trama original:  w1=0x1234  w2=0x5678  w3=0x9ABC  w4=0xDEF0
Trama con dos words intercambiadas:
                 w1=0x5678  w2=0x1234  w3=0x9ABC  w4=0xDEF0

Suma original      : 4660+22136+39612+57072 = 123480  →  → 57945
Suma intercambiada : 22136+4660+39612+57072  = 123480  →  → 57945

LAS SUMAS SON IDÉNTICAS → el checksum es el mismo
→ el receptor ACEPTA la trama.  El error pasa inadvertido.  ✗
```

Interpretación. Es **exactamente el mismo argumento de la paridad**, aplicado a words: dos perturbaciones que se compensan en la suma no dejan rastro. Por eso el checksum de UDP no protege contra reordenamientos de datos.

2.9 Ejercicio 8 (inventado) — CRC por división polinómica: `G = 1101`, `D = 100100`

Datos.

```
G = 1101 = x³ + x² + 1          r = 3
D = 100100 = x⁵ + x²
T = D·x³ = 100100000 = x⁸ + x⁵        (9 bits)
```

División polinómica, paso a paso. En cada paso, `G` cubre las **cuatro columnas que empiezan en la del término mayor**, con los valores `d₃ d₂ d₁ d₀`:

```
Paso 1    1 0 0 1 0 0 0 0 0
        ⊕ 1 1 0 1 0 0 0 0 0      ← G en x⁸
          ─────────────────
          0 1 0 0 0 0 0 0 0

Paso 2    0 1 0 0 0 0 0 0 0
        ⊕ 0 1 1 0 1 0 0 0 0      ← G en x⁷
          ─────────────────
          0 0 1 0 1 0 0 0 0

Paso 3    0 0 1 0 1 0 0 0 0
        ⊕ 0 0 1 1 0 1 0 0 0      ← G en x⁶
          ─────────────────
          0 0 0 1 1 1 0 0 0

Paso 4    0 0 0 1 1 1 0 0 0
        ⊕ 0 0 0 1 1 0 1 0 0      ← G en x⁵
          ─────────────────
          0 0 0 0 0 1 0 1 0

Paso 5    0 0 0 0 0 1 0 1 0
        ⊕ 0 0 0 0 0 1 1 0 1      ← G en x³
          ─────────────────
          0 0 0 0 0 0 0 0 1
                                        grado 0 < 3 → FIN
```

Respuesta

```
R = 001  (3 bits = el grado de G)
Se transmite:  100100 001
Recepción:     100100001 ÷ 1101 = 1011110 , resto 0
```

Verificación por multiplicación en `GF(2)` (que es la única forma válida: en `GF(2)` multiplicar **no** es multiplicar enteros, es XOR de desplazamientos):

```
Q = 1011110
Q << 3 = 1 0 1 1 1 1 0 0 0     (posiciones 8..0: 1 1 1 1)
Q << 2 =   1 0 1 1 1 1 0 0     (posiciones 7..0: 1 1 1 1)
Q       =       1 0 1 1 1 0     (posiciones 5..0: 1 1 1 1 0 1)

XOR columna por columna:
   pos8: 1                  = 1
   pos7: 1 ⊕ 1               = 0
   pos6: 1 ⊕ 1               = 0
   pos5: 1 ⊕ 1 ⊕ 1           = 1
   pos4:     1 ⊕ 1           = 0
   pos3: 1 ⊕ 1               = 0
   pos2:     1 ⊕ 1           = 0
   pos1:         0 ⊕ 1       = 1
   pos0:         1           = 1
──────────────────────────
   resultado = 1 0 0 1 0 0 0 0 1  = 100100001  ✓
```

⚠️ Errores típicos

- **Perder la cuenta de columnas** al escribir un XOR largo. El error más común de todos: uno escribe la fila del generador con un cero de más o de menos y el resultado sale distinto. **Por eso en el examen hay que alinear las columnas con espacios**, y **por eso en la práctica se usa el registro de desplazamiento** (§2.10), que sólo toca `r` bits por iteración y es inmune a este error.
- **Sumar en vez de XORar.** En `GF(2)` no hay acarreo.
- **Olvidar los `r` ceros a la derecha.** Sin ellos el polinomio se divide mal y el receptor no puede recomputar el resto.

2.10 Ejercicio 9 (inventado) — El mismo cálculo con la tabla de XOR

Datos. `G = 1011 = x³ + x + 1`, `r = 3`, `G mod 2³ = 011`. `D = 1001001 = x⁶ + x³ + 1`.

Método del registro de desplazamiento (§1.5.3), un renglón por bit de `D`:

paso	b	R	MSB(R)⊕b	R = R<<1	¿XOR 011?	R final
1	1	000	1	000	sí	011
2	0	011	0	110	no	110
3	0	110	1	100	sí	111
4	1	111	0	110	no	110
5	0	110	1	100	sí	111
6	0	111	1	110	sí	101
7	1	101	0	010	no	010
						R = 010

Y la división polinómica del mismo caso, para que se vean iguales:

```
T = D·x³ = 1001001000 = x⁹ + x⁶ + x³      (10 bits)

Paso 1    1 0 0 1 0 0 1 0 0 0
        ⊕ 1 0 1 1 0 0 0 0 0 0      ← G en x⁹
          ─────────────────────
          0 0 1 0 0 0 1 0 0 0

Paso 2    0 0 1 0 0 0 1 0 0 0
        ⊕ 0 0 1 0 1 1 0 0 0 0      ← G en x⁷
          ─────────────────────
          0 0 0 0 1 1 0 0 0 0

Paso 3    0 0 0 0 1 1 0 0 0 0
        ⊕ 0 0 0 0 1 0 1 1 0 0      ← G en x⁵
          ─────────────────────
          0 0 0 0 0 1 1 1 0 0

Paso 4    0 0 0 0 0 1 1 1 0 0
        ⊕ 0 0 0 0 0 1 0 1 1 0      ← G en x⁴
          ─────────────────────
          0 0 0 0 0 0 1 0 1 0

Paso 5    0 0 0 0 0 0 1 0 1 0
        ⊕ 0 0 0 0 0 0 0 1 0 1 1    ← G en x³ (las 4 últimas columnas)
          ─────────────────────
          0 0 0 0 0 0 0 0 1 0      ← grado 1 < 3 → FIN
```

```
R = 010   ✓  el mismo que por la tabla de XOR

Verificación:  T ⊕ R = 1001001000 ⊕ 0000000010 = 1001001010
               Q = 1011110 y  Q × 1011 = 1001001010   (resto 0)
```

Interpretación. Dos métodos, un resultado. El de la **división polinómica** es el que hay que saber hacer en un examen de papel; el del **registro de desplazamiento** es el que está en el silicio y el que usan `zlib`, `iSCSI`, SCTP y la placa de red de tu máquina.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-02, Pregunta 3 (8 puntos) — paridad en dos dimensiones

> **Fuente:** Examen `eirc1002` / `sirc1002` — febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf

Enunciado (texto del examen)

> Pregunta 3 (8 puntos)
>
> a) Explique el funcionamiento del mecanismo de chequeo por paridad en dos dimensiones utilizado para detección de errores en Capa de Enlace.
>
> b) ¿Permite realizar correcciones? ¿De cuántos bits? Justifique

Respuesta oficial (del PDF de soluciones)

> La paridad en dos dimensiones considera el flujo de datos como un flujo "bi dimensional", tomando el flujo como una matriz donde cada palabra es puesta en una fila, y palabras consecutivas en filas consecutivas. De esta forma el bit j-ésimo de cada palabra forma la columna j-ésima de toda la matriz.
>
> A cada fila se le agrega un bit de paridad (calculado sobre la fila) y cada cierto numero de palabras se agrega una palabra de paridad calculada por columna.
>
> La paridad en dos dimensiones permite: - Corregir errores de un bit por palabra y por columna, ya que esto hace posible saber que bit está invertido a través de la intersección de la fila y columna que fallan la verificación de paridad. - En algunos casos permite detectar errores dobles.

Análisis de la solución oficial

Lo que está bien, y es el corazón del puntaje:

1. **La definición de la matriz** es exactamente la que hay que usar: "cada palabra es puesta en una fila, palabras consecutivas en filas consecutivas, el bit j-ésimo de cada palabra forma la columna j-ésima". Con eso se entiende que **no hay ambigüedad** en cómo se arma.
2. **"Una palabra de paridad calculada por columna"** — ojo con la frase: la solución dice "**palabra**" de paridad por columna, cuando lo correcto es que el cálculo de columna **escribe un bit por columna**, todos en una última fila. Las dos cosas son consistentes (una fila de bits de paridad), pero si el enunciado mezclara los niveles, hay que aclarar cuál es cuál.

3. **La razón de la corrección** está bien: "*a través de la intersección de la fila y columna que fallan la verificación de paridad*". Ésa es la frase que hay que decir.

Lo que le falta — y son los puntos que se pueden sumar:

1. **El ítem (b) pide "de cuántos bits", y la solución oficial no lo dice con claridad.** Hay que decir explícitamente: **corrige exactamente 1 bit**.
2. **"En algunos casos permite detectar errores dobles"** es **demasiado generoso**. La 2D paridad detecta **todos** los errores de dos bits, no "algunos". Lo que es cierto es que **no puede corregirlos**.
3. **No menciona el techo**: existen errores de **cuatro** bits (un rectángulo `2 × 2`) que **no se detectan en absoluto**. Es el dato que demuestra que entendiste la estructura lineal del código.
4. **No justifica por qué 2D y no 1D.** La respuesta es: 1D detecta los errores de **número impar** de bits y ninguno de número par; 2D agrega la dimensión que faltaba.

> **Nota:** la solución oficial del ítem (b) es más laxa que lo que la pregunta pide. Mi respuesta es más fuerte, y creo que es la que el corrector busca: la pregunta dice "¿de cuántos bits?" y eso exige un número.

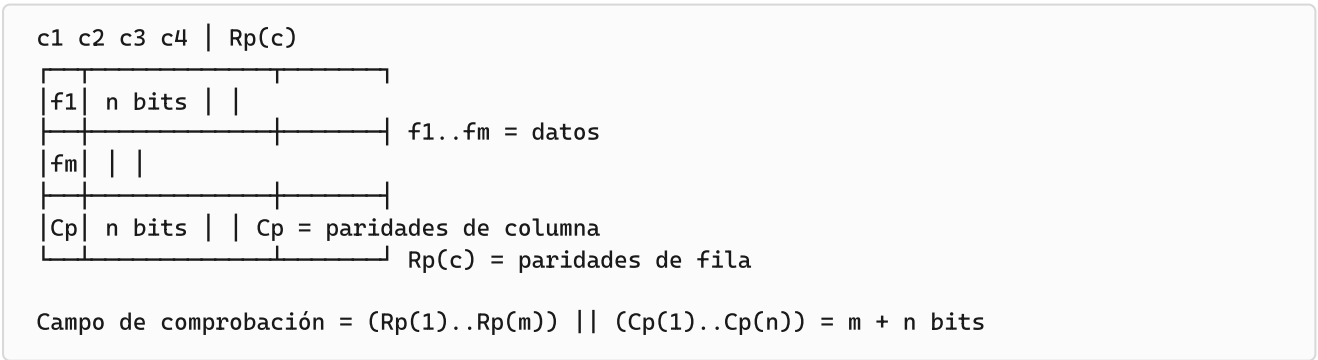
Mi resolución

(a) Cómo funciona.

Se considera el flujo de datos como **una matriz de `m` filas por `n` columnas**: cada palabra de `n` bits del flujo se coloca en una fila, y palabras consecutivas en filas consecutivas. Así, **el bit `j` de cada palabra forma la columna `j`** de toda la matriz.

Al bloque se le agrega **un campo de comprobación de `m + n` bits**: - **un bit de paridad al final de cada fila**, calculado sobre los `n` bits de esa fila; - **un bit de paridad al final de cada columna**, calculado sobre los `m` bits de esa columna.

El receptor **recalcula las `m` paridades de fila y las `n` paridades de columna** y las compara con las recibidas:



Nota: no hace falta un bit de esquina. La paridad de la suma de las paridades de fila y la de la suma de las de columna son **iguales** por construcción (ambas son la paridad del número total de unos de la matriz), así que la esquina es redundante.

(b) ¿Corrige? ¿Cuántos bits?

Sí: corrige exactamente 1 bit. Y detecta (sin corregir) **todos** los errores de dos bits.

Justificación de la corrección de 1 bit:

```
Si hay UN solo bit equivocado, y está en la fila i, columna j:

→ la paridad de la FILA i falla (tiene un uno de más o de menos)
→ la paridad de la COLUMNA j falla (idem)

→ la intersección (i, j) queda completamente determinada
→ el receptor invierte ese bit y el mensaje queda EXACTO.
```

Justificación de por qué no puede corregir 2 bits:

```
Con dos errores, hay varios casos:

Caso 1 — en filas distintas y columnas distintas: (i,j) y (k,l), i≠k, j≠l
fallan las filas i y k, y las columnas j y l
→ 4 posiciones candidatas, sólo 2 están mal → INDETERMINADO

Caso 2 — en la misma columna: (i,j) y (k,j)
fallan las filas i y k; la columna j NO falla (2 errores se compensan)
→ 2 filas y 0 columnas: el patrón es imposible de atribuir

Caso 3 — en la misma fila: análogo, 0 filas y 2 columnas

En todos los casos: DETECTA que hay error (algo falló), pero NO PUEDE
decir dónde. Faltaría redundancia extra para desempatar.
```

El techo, que es lo que demuestra que entendiste:

```
Con un error de CUATRO bits formando un rectángulo 2×2 — (i,j), (i,l), (k,j),
(k,l) — cada fila y cada columna afectada recibe DOS cambios, de signo opuesto,
así que NINGUNA paridad se altera:

fila i: +1 y −1 → la paridad no cambia
fila k: +1 y −1 → la paridad no cambia
columna j: +1 y −1 → la paridad no cambia
columna l: +1 y −1 → la paridad no cambia

∴ Las 2D paridades NO detectan ese error. Cero verificaciones fallan.
```

Conclusión con la distancia de Hamming:

```
La 2D paridad es un código con d = 3 sobre los bits de datos
(d = 3 por el patrón de 3 flips en el bloque de paridades; y los patrones
de 2 flips en datos siempre se detectan).

Con d = 3:
detecta todos los errores de peso ≤ 2
corrige todos los errores de peso ≤ ⌊(3−1)/2⌋ = 1

Coincide con lo demostrado arriba. ✓
```

> **Tope para 8 puntos:** la definición de la matriz (palabras en filas, bit j = columna j), el campo de `m + n` bits, el recálculo de las paridades de fila y de columna, y la corrección del error de 1 bit **por**

> **intersección**. Para el ítem (b): decir explícitamente **"corrige 1 bit"** y **justificar por qué 2 no** (ambigüedad de las 4 intersecciones). Sumar el argumento del rectángulo 2 × 2 como techo.

PARTE 4 — CIERRE

📌 Resumen

1. **Detectar** es saber que hubo error; **corregir** es saber cuál era el bit y repararlo. El corrector cuesta mucha más redundancia.
2. **Paridad 1D**: un bit. Detecta **todos** los errores de **número impar** de bits y **ninguno de número par** (dos bits que se voltean se compensan).
3. **Paridad 2D**: matriz `m × n` + `m` paridades de fila + `n` de columna. Campo de `m + n` bits. La esquina es redundante.
4. **Corrige 1 bit** (por intersección fila × columna) y **detecta todos los de 2**, pero **no los corrige**. Hay un techo: el **rectángulo 2 × 2** de 4 errores **no se detecta**.
5. **Internet checksum**: 16 bits, complemento a uno, con acarreo circular. Detecta todos los errores de un bit; **no detecta el intercambio de dos words** (la suma es conmutativa).
6. **CRC**: división polinómica en `GF(2)`, donde `+` es **XOR** y `×` es **AND**. Se agregan `r` ceros y se divide; el resto es el CRC.
7. **El hardware** hace lo mismo con un **registro de `r` bits y un XOR por bit**: los dos métodos dan el mismo resultado.
8. **CRC-32 = `0x04C11DB7`** (Ethernet, ZIP, PNG), **CRC-32C = `0x1EDC6F41`** (SCTP, iSCSI), **CRC-16/CCITT = `0x1021`** (HDLC). Hay varios porque cada aplicación optimiza para algo distinto.
9. **Propiedad clave**: un error pasa inadvertido **si y sólo si es múltiplo de `G`**. Por eso el CRC detecta **siempre** un error de 1 bit y **siempre** una ráfaga de hasta `r` bits. La distancia mínima es `d = r + 1`.
10. **El CRC detecta, no corrige.** Para corregir hacen falta **Hamming** o **Reed–Solomon**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Demostrá por qué la paridad de una dimensión **no detecta** un error de dos bits.

<details>
<summary>Respuesta</summary>
</details>

2. En una matriz 4 × 4 con paridad par, ¿cuánto mide el campo de comprobación y por qué no hace falta el bit de esquina?

<details>
<summary>Respuesta</summary>
</details>

3. Se detecta un error de un bit y fallan la fila 2 y la columna 5. ¿Qué hacés y qué se corrige?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué dos errores no se pueden corregir con paridad 2D, si los errores sí se detectan?

▶ Respuesta

5. ¿Por qué el Internet checksum **no detecta** el intercambio de dos words de 16 bits? ¿Qué otro error tampoco detecta?

▶ Respuesta

6. En `GF(2)`, ¿qué son la suma y el producto? ¿Por qué se usan polinomios y no división entera?

▶ Respuesta

7. Ejecutá el CRC de `D = 10101010` con `G = 1001`. ¿Cuál es `R` y cuánto overhead agrega?

▶ Respuesta

8. ¿Por qué hay varios polinomios generadores de CRC estandarizados en lugar de uno solo?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1002` / `sirc1002`** — P3 (8 pts, paridad en dos dimensiones: funcionamiento y corrección)
- **Examen `eirc0707` / `sirc0707`** — P2 (5 pts, nombres de los mecanismos de detección y corrección de la capa de enlace)
- **Examen `eirc0812` / `sirc0812`** — P4 (7 pts, DLE en POP3: el mismo concepto de transparencia)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 4 §4.2 ("Error Detection and Correction")
- **Kurose & Ross**, *Computer Networking: A Top-Down Approach*, §5.5 ("Error Detection and Correction") — figuras 5.4 a 5.9
- **Práctico 5 de 2008** — `material-oficial/practicos/png/prac05_2008-p01.png`, ejercicios 1 a 4 (paridad 2D, Internet checksum, CRC)
- `RFC 1071` — Internet checksum: cálculo en UDP e IP
- `RFC 1624` — por qué no se puede cambiar el algoritmo del checksum de TCP/UDP
- `RFC 9293` — TCP (2022, *Internet Standard*): el checksum de TCP es el Internet checksum
- **IEEE 802.3** — el FCS de Ethernet es un CRC-32 (`0x04C11DB7`), 4 bytes
- **Clase siguiente:** 40 — Corrección de errores y ARQ: Hamming, Stop-and-Wait, Go-Back-N y Selective Repeat